CHAPTER 1

Language model evaluation, metrics, and benchmarks

The pre-training paradigm detailed in Chapter **??** established that scaling parameter count, dataset volume, and training compute yields continuous, predictable reductions in next-token cross-entropy loss. By optimizing this autoregressive objective across trillions of tokens, modern foundation models acquire rich linguistic representations, syntactic abstractions, and emergent in-context learning capabilities. However, a fundamental discrepancy exists between pre-training optimization and practical deployment: minimizing the empirical cross-entropy loss over a corpus merely measures the statistical predictability of tokens under teacher forcing. It does not directly quantify whether an autoregressively generated sequence is coherent, factually truthful, stylistically appropriate, or capable of solving complex multi-step reasoning tasks.

Evaluating natural language generation is fundamentally challenging because open-ended text does not conform to the closed-world assumptions of classical machine learning. In standard classification or regression tasks, the output space is bounded and discrete labels provide unambiguous ground truth. In contrast, generative language modeling is combinatorial: a single semantic concept, question prompt, or code specification can be articulated through an astronomical number of valid, syntactically diverse surface formulations. Conversely, a generated string may match reference vocabulary with high lexical overlap while completely inverting factual truth through the addition of a single negation token. Qualitative inspection—relying on human observers to informally assess whether sampled text appears coherent or degraded—is unscalable, subjective, and unable to establish statistically sound comparisons between competing architectures or training checkpoints.

To establish rigorous engineering standards, the natural language processing community has developed an evaluation hierarchy that spans multiple levels of abstraction. At the lowest level lie intrinsic probabilistic metrics, such as cross-entropy and perplexity, which evaluate the model's internal likelihood assignments directly on held-out text without requiring generated outputs or human task labels. At the intermediate level lie reference-based lexical metrics, such as BLEU and ROUGE, which compare model-generated candidate sequences against human reference strings using n-gram overlap and length penalties. At higher levels of the hierarchy, evaluation shifts toward contextual embedding similarity (such as BERTScore), model-based evaluators, and standardized task benchmarks (ranging from core natural language understanding suites like GLUE to comprehensive multi-task and code-generation suites like MMLU and HumanEval).

This chapter formalizes the principles of language model evaluation. We begin by examining the core mathematical properties required for rigorous evaluation metrics, focusing on the distinction between additive and multiplicative averaging, before conducting an exhaustive investigation of intrinsic uncertainty estimation via perplexity and surface-string reference matching via BLEU.

1.1 Evaluation metrics for language models

Evaluation metrics in natural language processing can be partitioned into two distinct paradigms: intrinsic likelihood metrics and extrinsic generation metrics. Intrinsic metrics evaluate the model's mathematical representation of language by computing the probability distribution it assigns to unseen test corpora under teacher forcing. They require no sample generation, no decoding algorithms, and no reference texts; they assess the predictive confidence of the probability model itself. Extrinsic metrics, by contrast, operate on the discrete sequences emitted during autoregressive inference, comparing the generated hypothesis against one or more human gold-standard references.

The geometric mean and multiplicative error penalization.

Before formalizing specific metrics, we must establish the mathematical foundation of sequence-level score aggregation. When evaluating models across multiple tokens, sentences, or n-gram orders, individual scores must be aggregated into a single scalar summary. While the standard arithmetic mean is familiar, it possesses structural properties that render it unsuitable for probability spaces and multi-criteria quality evaluation.

Let $\mathbf{x} = (x_1, x_2, \ldots, x_n) \in \mathbb{R}^n_{>0}$ denote a collection of n positive evaluation values (such as token probabilities or n-gram precision scores). The arithmetic mean (AM) and geometric mean (GM) are defined as:

$$\mathrm{AM}(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} x_i, \tag{1.1}$$

$$\mathrm{GM}(\mathbf{x}) = \left(\prod_{i=1}^{n} x_i\right)^{\frac{1}{n}}. \tag{1.2}$$

The critical distinction between these two operators lies in how they penalize low values. The arithmetic mean exerts an *additive effect*: a severely low score in one component can be easily masked by high scores in other components. For example, consider three evaluation scores $\mathbf{x} = (0.1, 0.9, 0.9)$. The arithmetic mean yields:

$$\mathrm{AM}(0.1, 0.9, 0.9) = \frac{0.1 + 0.9 + 0.9}{3} = \frac{1.9}{3} \approx 0.6333, \tag{1.3}$$

producing a deceptively favorable aggregate score despite catastrophic failure on the first criterion. In contrast, the geometric mean exerts a *multiplicative effect*:

$$\mathrm{GM}(0.1, 0.9, 0.9) = (0.1 \times 0.9 \times 0.9)^{\frac{1}{3}} = (0.081)^{\frac{1}{3}} \approx 0.4327. \tag{1.4}$$

Because the terms multiply, any degradation in an individual component disproportionately suppresses the entire product. Crucially, if any single term reaches zero ($x_i = 0$), the geometric mean collapses to zero:

$$\exists i \text{ such that } x_i = 0 \implies \mathrm{GM}(\mathbf{x}) = 0. \tag{1.5}$$

This property is indispensable in evaluation: a model must achieve reasonable competence across all evaluated dimensions to attain a high overall score. This behavior directly parallels the harmonic mean used in the F_1-score to balance precision and recall.

Numerical stability via the logarithmic formulation.

In practical computation, evaluating long sequences or high-order n-grams involves multiplying dozens or hundreds of terms bounded in $(0, 1]$. In finite-precision floating-point arithmetic (IEEE 754 float32 or float64), direct repeated multiplication inevitably causes arithmetic underflow, rounding the running product to absolute zero.

To guarantee numerical stability, the geometric mean is computed in the logarithmic domain. Utilizing the monotonic identity $\ln\left(\prod x_i\right) = \sum \ln x_i$, the n-th root is transformed into an arithmetic average of logarithms:

$$\mathrm{GM}(\mathbf{x}) = \exp\left(\ln\left(\prod_{i=1}^{n} x_i\right)^{\frac{1}{n}}\right) = \exp\left(\frac{1}{n}\sum_{i=1}^{n} \ln x_i\right). \tag{1.6}$$

As emphasized in lecture, the base of the logarithm must strictly match the base of the exponentiation operator. Standard scientific implementations employ the natural logarithm ln (base e) and natural exponential $\exp(\cdot)$ so that $\exp(\ln(z)) = z$ is an exact inverse mapping. If base-2 logarithms ($\log_2$) are adopted, the outer transformation must evaluate as $2^{\frac{1}{n}\sum \log_2 x_i}$.

1.1.1 Perplexity

The primary intrinsic metric for evaluating autoregressive language models is **perplexity** (PPL). Rooted in Claude Shannon's mathematical theory of communication and information entropy [**?**], perplexity quantifies the degree of uncertainty a probabilistic model exhibits when predicting a sequence of tokens.

Conceptual intuition and the effective branching factor.

Rather than assessing whether a model generates fluent answers, perplexity measures how surprised the model is when presented with human-written, grammatically correct text. If a model has acquired an accurate statistical representation of English, it will assign high probability to legitimate sentences from held-out corpora. Conversely, if the model is poorly trained or faces out-of-domain text, it will assign low probability to valid tokens, indicating substantial confusion.

Perplexity provides an intuitive geometric interpretation as the **effective branching factor** of the language model. Suppose that at a specific step in a sentence, the model assigns probability $P(w_i \mid w_{<i})$ to the ground-truth token. The reciprocal:

$$K_i = \frac{1}{P(w_i \mid w_{<i})} \tag{1.7}$$

represents the number of equally probable alternative words among which the model was undecided.

Consider a thought experiment where the model's vocabulary contains 50,000 discrete tokens:

- If the model assigns probability $P = 0.25$ to the correct token, $\frac{1}{0.25} = 4$. Even though the vocabulary contains 50,000 candidates, the model's uncertainty is equivalent to rolling a fair 4-sided die of equally likely tokens.
- If the model assigns probability $P = 0.90$, $\frac{1}{0.90} \approx 1.11$. The model is near-certain of the true token, undecided between approximately 1.1 alternatives.
- If the model assigns probability $P = 1.0$, $\frac{1}{1.0} = 1.0$. The model has zero uncertainty.
- If the model assigns probability $P = 0.01$, $\frac{1}{0.01} = 100$. The model is as confused as choosing uniformly among 100 distinct words.

Across an entire sequence of N words, perplexity represents the geometric average of these individual branching factors. A perplexity of 50 indicates that, on average across each position in the sentence, the model was as uncertain as choosing uniformly among 50 equally probable words. Lower perplexity reflects greater predictive confidence and superior alignment with the underlying data distribution.

Formal mathematical derivation and the negative log-likelihood.
Let $W = (w_1, w_2, \ldots, w_N)$ denote a sequence of N tokens. By the probability chain rule, the joint probability assigned to sequence W by an autoregressive language model parameterized by θ is:

$$P(W) = \prod_{i=1}^{N} P(w_i \mid w_1, \ldots, w_{i-1}; \theta) = \prod_{i=1}^{N} P(w_i \mid w_{<i}). \tag{1.8}$$

The perplexity of the model over the sequence W is formally defined as the reciprocal of the geometric mean of the token conditional probabilities:

$$\mathrm{PPL}(W) = P(W)^{-\frac{1}{N}} = \left(\prod_{i=1}^{N} P(w_i \mid w_{<i}) \right)^{-\frac{1}{N}} = \left(\prod_{i=1}^{N} \frac{1}{P(w_i \mid w_{<i})} \right)^{\frac{1}{N}}. \tag{1.9}$$

Applying the logarithmic transformation in Equation (1.6), we rewrite perplexity in terms of the sum of token negative log-likelihoods (NLL):

$$\ln \mathrm{PPL}(W) = -\frac{1}{N} \ln P(W) = -\frac{1}{N} \sum_{i=1}^{N} \ln P(w_i \mid w_{<i}), \tag{1.10}$$

$$\mathrm{PPL}(W) = \exp\left(-\frac{1}{N} \sum_{i=1}^{N} \ln P(w_i \mid w_{<i}) \right) = \exp\left(\frac{1}{N} \sum_{i=1}^{N} \mathrm{NLL}_i \right), \tag{1.11}$$

where $\mathrm{NLL}_i = -\ln P(w_i \mid w_{<i}) \geq 0$ is the negative log-likelihood of token w_i.

Equation (1.11) reveals the direct mathematical link between perplexity and the pre-training cross-entropy loss $\mathcal{L}_{\mathrm{CE}}$:

$$\mathcal{L}_{\mathrm{CE}} = \frac{1}{N} \sum_{i=1}^{N} \mathrm{NLL}_i \implies \mathrm{PPL}(W) = e^{\mathcal{L}_{\mathrm{CE}}}. \tag{1.12}$$

Perplexity is simply the exponentiated cross-entropy loss. Figure **??** illustrates the exponential growth of perplexity as average cross-entropy loss increases, underscoring the disproportionately severe penalty imposed on uncertain predictions.

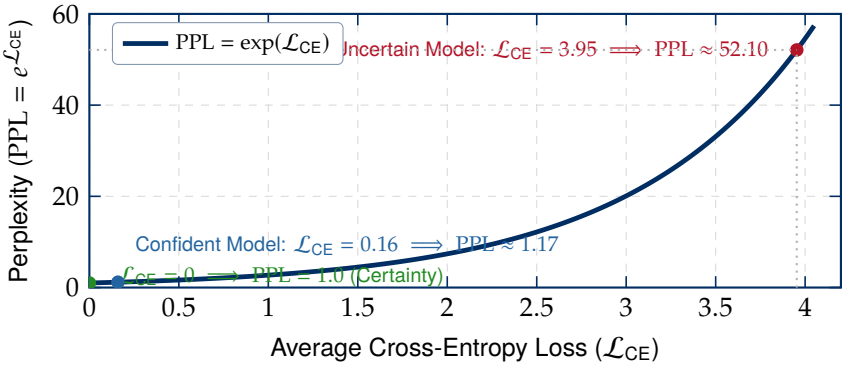


Figure 1.1: Perplexity (PPL) as an exponential function of the average cross-entropy loss $\mathcal{L}_{\mathrm{CE}}$. The metric attains its theoretical minimum at PPL = 1.0 when $\mathcal{L}_{\mathrm{CE}} = 0$. Due to the exponential mapping $\mathrm{PPL} = e^{\mathcal{L}_{\mathrm{CE}}}$, modest increments in cross-entropy loss trigger dramatic escalations in perplexity, effectively penalizing poor probability assignments.

Mathematical justification for the geometric mean. The selection of the geometric mean in Equation (1.9) is mathematically mandatory for three reasons:

1. **Consistency with probability factorization**: In probability theory, the likelihood of a sequence is formed by the product of conditional probabilities, not their sum. Normalizing a product of N terms by sequence length requires taking the N-th root, so that $\text{PPL}(W)^N = \frac{1}{P(W)}$. Taking an arithmetic mean of probabilities ($\frac{1}{N}\sum P_i$) violates the product rule and lacks theoretical interpretation.

2. **Information-theoretic duality**: Taking the logarithm of the geometric mean yields the arithmetic mean of negative log-likelihoods (cross-entropy), establishing an exact duality with Shannon entropy. An arithmetic mean of probabilities cannot be converted into cross-entropy.

3. **Zero-frequency penalty**: If a model assigns zero probability to a single valid token ($P(w_k \mid w_{<k}) = 0$), the geometric mean product collapses to 0, which inverts to infinite perplexity: $\text{PPL}(W) \to \infty$. This correctly identifies the model as broken, since it declared a real-world sentence impossible. An arithmetic mean would merely incur a negligible $\frac{1}{N}$ reduction.

The theoretical lower bound of perplexity is PPL = 1.0. This minimum occurs if and only if the model assigns probability $P(w_i \mid w_{<i}) = 1.0$ to every token in the sequence ($\text{NLL}_i = 0 \implies e^0 = 1$), representing absolute, deterministic certainty. In practice, PPL = 1.0 is observed only when a model severely overfits a training sequence.

Step-by-step worked examples: confident versus uncertain models.

To illustrate the mechanics of perplexity, we walk through the exact numerical evaluations presented in the lecture. Consider a held-out test sentence delimited by boundary markers:

$$W = \langle\text{bos}\rangle\ \text{The dog chased the cat}\ \langle\text{eos}\rangle.$$

The beginning-of-sequence token ⟨bos⟩ serves as initial conditioning prefix ($P = 1.0$). The sequence contains $N = 6$ evaluated transitions ($w_1 = \text{The}, \ldots, w_6 = \langle\text{eos}\rangle$).

Case 1: Confident, well-trained model. Suppose a model assigns high probability to each correct token, reflecting strong linguistic familiarity. The conditional probabilities, negative log-likelihoods, and equivalent branching numbers are listed in Table 1.1.

Table 1.1: Step-by-step perplexity computation for a confident model on $W = \langle\text{bos}\rangle$ The dog chased the cat ⟨eos⟩.

i	Token w_i	Probability $P(w_i \mid w_{<i})$	$\text{NLL}_i = -\ln P(w_i \mid w_{<i})$	Branching $K_i = \frac{1}{P_i}$
1	The	0.90	$-\ln(0.90) \approx 0.1054$	1.1111
2	dog	0.85	$-\ln(0.85) \approx 0.1625$	1.1765
3	chased	0.88	$-\ln(0.88) \approx 0.1278$	1.1364
4	the	0.75	$-\ln(0.75) \approx 0.2877$	1.3333
5	cat	0.95	$-\ln(0.95) \approx 0.0513$	1.0526
6	⟨eos⟩	0.80	$-\ln(0.80) \approx 0.2231$	1.2500
Sum / Mean		—	$\sum \text{NLL}_i = 0.9578 \implies \overline{\text{NLL}} = 0.1596$	$\prod K_i \approx 2.6074$

Evaluating the perplexity via the average negative log-likelihood:

$$\begin{aligned}\text{PPL}(W) &= \exp\left(\frac{1}{6}\sum_{i=1}^{6}\text{NLL}_i\right)\\ &= \exp\left(\frac{0.1054 + 0.1625 + 0.1278 + 0.2877 + 0.0513 + 0.2231}{6}\right)\\ &= e^{0.15963} \approx \mathbf{1.1731}.\end{aligned} \tag{1.13}$$

Equivalently, taking the 6-th root of the product of equivalent words:

$$\text{PPL}(W) = (1.1111 \times 1.1765 \times 1.1364 \times 1.3333 \times 1.0526 \times 1.2500)^{\frac{1}{6}} \approx (2.6074)^{\frac{1}{6}} \approx \mathbf{1.1731}. \quad (1.14)$$

The model's perplexity is 1.1731, indicating that on average, the model was undecided over approximately 1.17 equivalent words per position. It possessed near-complete certainty throughout the sequence.

Case 2: Uncertain, poorly trained model. Now suppose an under-trained or out-of-domain model evaluates the identical sentence, assigning substantially lower probabilities to the correct tokens, as detailed in Table 1.2.

Table 1.2: Step-by-step perplexity computation for an uncertain model on $W = \langle\text{bos}\rangle$ The dog chased the cat $\langle\text{eos}\rangle$.

i	Token w_i	Probability $P(w_i \mid w_{<i})$	$\text{NLL}_i = -\ln P(w_i \mid w_{<i})$	Branching $K_i = \frac{1}{P_i}$
1	The	0.050	$-\ln(0.050) \approx 2.9957$	20
2	dog	0.100	$-\ln(0.100) \approx 2.3026$	10
3	chased	0.200	$-\ln(0.200) \approx 1.6094$	5
4	the	0.010	$-\ln(0.010) \approx 4.6052$	100
5	cat	0.005	$-\ln(0.005) \approx 5.2983$	200
6	$\langle\text{eos}\rangle$	0.001	$-\ln(0.001) \approx 6.9078$	1000
Sum / Product	—	$\sum \text{NLL}_i = 23.7190 \implies \overline{\text{NLL}} = 3.9532$		$\prod K_i = 2 \times 10^{10}$

Computing the geometric mean directly from the branching factors:

$$\prod_{i=1}^{6} K_i = 20 \times 10 \times 5 \times 100 \times 200 \times 1000 = 2 \times 10^{10}, \quad (1.15)$$

$$\text{PPL}(W) = \left(2 \times 10^{10}\right)^{\frac{1}{6}} \approx \mathbf{52.10}. \quad (1.16)$$

Verifying via the average negative log-likelihood:

$$\text{PPL}(W) = \exp\left(\frac{23.7190}{6}\right) = \exp(3.95317) \approx \mathbf{52.10}. \quad (1.17)$$

Both formulations yield identical results: PPL = 52.10. On average, the uncertain model was confused across approximately 52 equally likely candidate tokens at every position.

As highlighted by the professor, perplexity serves as a diagnostic probe into the temporal and thematic distribution of a model's pre-training data. If an older model trained on corpora prior to 2016 is evaluated on the prompt "The President of the United States is [NAME]", evaluating "Barack Obama" yields low perplexity, whereas evaluating "Donald Trump" or "Joe Biden" produces high perplexity. The elevated perplexity does not imply that the model is architecturally defective; rather, it indicates statistical surprise relative to the pre-training data distribution.

Advantages and fundamental limitations of perplexity.

Perplexity possesses distinct methodological advantages that have cemented its status as standard reporting metric in pre-training literature:

- **Unsupervised and task-agnostic**: Computing perplexity requires only raw, unannotated text. It eliminates the need for expensive human reference translations, task labels, or subjective grading rubrics.
- **Zero inference compute overhead**: Because evaluation occurs under teacher forcing, perplexity is computed in a single forward pass over batches of tokens simultaneously with cross-entropy training loss, avoiding autoregressive generation.

- **Continuous and sensitive**: Perplexity is a smooth, continuous scalar that detects subtle improvements in language modeling capability long before downstream task accuracies show measurable gains.

Despite these strengths, perplexity suffers from fundamental limitations that prevent it from serving as a universal measure of model quality:

- **Vocabulary and tokenizer incomparability**: Perplexity is strictly incomparable across models that use different vocabularies or tokenization algorithms. Consider a byte-level tokenizer ($|\mathcal{V}| = 256$) versus a subword tokenizer ($|\mathcal{V}| = 100,000$). Because a byte-level model predicts tiny, predictable character fragments, its per-token perplexity will appear dramatically lower than the subword model, even though the subword model generates superior semantic text. Valid cross-model comparisons require normalizing perplexity by the number of words rather than subword tokens:

$$\text{PPL}_{\text{word}} = \exp\left(\frac{N_{\text{tokens}}}{N_{\text{words}}}\mathcal{L}_{\text{CE}}\right). \tag{1.18}$$

- **Disconnection from reasoning and factual truth**: A model can achieve outstanding (low) perplexity by mastering local grammatical syntax, common idioms, and surface transitions while remaining completely incapable of logical deduction, arithmetic calculation, or factual consistency. Perplexity rewards familiarity, not correctness.

1.1.2 Bilingual Evaluation Understudy (BLEU)

While perplexity evaluates the model's internal probability distribution over given text, practical deployment requires evaluating the actual text generated by the model during autonomous inference. In 2002, Papineni, Roukos, Ward, and Zhu [**?**] introduced the **Bilingual Evaluation Understudy** (BLEU) metric at IBM Research. Originally developed for machine translation, BLEU became the foundational reference-based evaluation metric across natural language processing.

Conceptual intuition and reference-based lexical matching.

The fundamental premise of BLEU is that the closer a machine-generated candidate translation resembles a human gold-standard translation, the higher its quality. Evaluating open-ended text against human references requires navigating two competing linguistic dimensions:

1. **Adequacy (Word Choice)**: Did the model translate the correct conceptual content? This is captured by measuring lexical overlap at the unigram level (1-grams).

2. **Fluency (Word Order)**: Did the model assemble the words in a natural, syntactically coherent sequence? This is captured by measuring contiguous overlap across higher-order n-grams (2-grams, 3-grams, 4-grams).

BLEU is an extrinsic metric: it requires running the model autoregressively to generate a hypothesis string G and comparing G against one or more reference strings $R = \{R_1, \ldots, R_m\}$. It is defined as a modified n-gram precision metric multiplied by an exponential brevity penalty.

Mathematical formulation and the brevity penalty.

To compute BLEU, we first extract all contiguous n-grams of size $i \in \{1, 2, \ldots, n\}$ from both the generated candidate sequence G and the reference sequence R.

Modified clipped n-gram precision. A naive precision metric would simply count the fraction of candidate n-grams that appear in the reference:

$$p_{\text{naive}} = \frac{\sum_{g \in G} \mathbb{I}(g \in R)}{|G|}.$$

However, naive precision can be exploited through pathological repetition. For example, if the reference is "The cat sat on the mat" and a defective model outputs the candidate "the the the the the the", every single generated unigram matches the reference word "the". Naive precision would reward this degenerative output with a perfect score: $p_{\text{naive}} = \frac{6}{6} = 1.0$.

To eliminate this vulnerability, Papineni et al. [**?**] introduced **clipped precision** (p_i). For each i-gram in the candidate sequence, the count is clipped so that it cannot exceed the maximum number of times that identical i-gram appears in any single reference sentence:

$$\text{Count}_{\text{clip}}(g_i) = \min\left(\text{Count}_G(g_i),\ \max_{R_j \in R} \text{Count}_{R_j}(g_i)\right). \tag{1.19}$$

The modified i-gram precision p_i is then computed as the sum of clipped counts divided by the total number of candidate i-grams generated:

$$p_i = \frac{\sum_{g_i \in G} \text{Count}_{\text{clip}}(g_i)}{\sum_{g_i \in G} \text{Count}_G(g_i)}. \tag{1.20}$$

In the pathological repetition example, the reference contains the unigram "the" twice ($\max \text{Count}_R(\text{"the"}) = 2$). The clipped count for the candidate is $\min(6, 2) = 2$, reducing the unigram precision to $\frac{2}{6} = 0.3333$.

Multi-order precision and the brevity penalty. While clipped precision prevents repetitive over-generation, precision alone introduces an opposite failure mode: **under-generation**.

Suppose the reference sentence contains 20 words, and the candidate model generates only two words: "The dog". Both words appear in the reference, so $p_1 = \frac{2}{2} = 1.0$. The bigram "The dog" also appears in the reference, so $p_2 = \frac{1}{1} = 1.0$. Without length regulation, the truncated output would achieve a flawless score: BLEU = 1.0, despite discarding 90% of the intended content.

Standard classification metrics resolve under-generation by computing recall. However, in machine translation and open-ended generation, defining recall is complicated because human translations exhibit natural length variance, and multiple reference translations often use different phrasing. Instead of explicit recall, BLEU introduces an exponential multiplicative **Brevity Penalty** (BP).

Let g denote the length of the generated candidate sequence, and let r denote the effective length of the reference sequence. The brevity penalty is defined as:

$$\text{BP} = \begin{cases} 1.0 & \text{if } g > r, \\ \exp\left(1 - \frac{r}{g}\right) & \text{if } g \leq r. \end{cases} \tag{1.21}$$

Figure 1.1 illustrates the mathematical behavior of the brevity penalty as a function of the length ratio $\frac{g}{r}$.

- When $g \geq r$, the candidate length equals or exceeds the reference length. In this regime, BP = 1.0: no brevity penalty is applied. Overly long generations are naturally penalized by the denominator of the precision terms in Equation (1.20).
- When $g < r$, the exponent $1 - \frac{r}{g}$ is strictly negative, causing BP to decay exponentially below 1.0. As the candidate length approaches zero ($g \to 0$), the ratio $\frac{r}{g} \to \infty$, and BP $\to 0$, aggressively driving the overall score to zero.

Formal mathematical derivation of the BLEU metric.
With the individual components defined, we now formally derive the composite BLEU formula from first principles, following the exact mathematical trajectory used for perplexity.

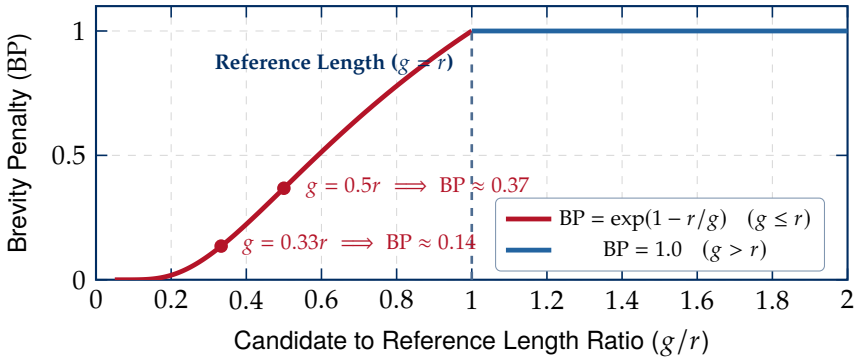


Figure 1.2: The BLEU Brevity Penalty (BP) as a function of the generated-to-reference length ratio g/r. For $g \geq r$, BP = 1.0 because over-generation is already penalized by precision denominators. For $g < r$, BP = $\exp(1 - r/g)$ decays exponentially toward zero, severely penalizing short or truncated generations to prevent precision gaming.

Let $\{p_1, p_2, \ldots, p_n\}$ denote the modified clipped precisions up to order n, with corresponding non-negative weights $w_i \geq 0$ satisfying $\sum_{i=1}^{n} w_i = 1$. The collective precision across all n-gram orders is combined using a weighted geometric mean:

$$\mathrm{GM}(p_1, \ldots, p_n) = \prod_{i=1}^{n} p_i^{w_i}. \tag{1.22}$$

Under standard uniform weighting across the n orders, $w_i = \frac{1}{n}$, simplifying the product to:

$$\mathrm{GM}(p_1, \ldots, p_n) = \left(\prod_{i=1}^{n} p_i \right)^{\frac{1}{n}}. \tag{1.23}$$

In computational implementations, multiplying small probabilities or fractional precisions directly induces numerical underflow. To guarantee numerical stability, we map the geometric mean into the log-domain:

$$\begin{aligned} \ln \mathrm{GM}(p_1, \ldots, p_n) &= \ln \left(\prod_{i=1}^{n} p_i^{w_i} \right) \\ &= \sum_{i=1}^{n} \ln \left(p_i^{w_i} \right) \\ &= \sum_{i=1}^{n} w_i \ln p_i. \end{aligned} \tag{1.24}$$

Exponentiating both sides restores the original geometric scale:

$$\mathrm{GM}(p_1, \ldots, p_n) = \exp \left(\sum_{i=1}^{n} w_i \ln p_i \right). \tag{1.25}$$

Notice the mathematical parallel with perplexity: just as perplexity maps the arithmetic mean of negative log-likelihoods back to probability space via $\exp(-\frac{1}{N} \sum \ln P_i)$, BLEU maps the weighted arithmetic mean of log-precisions back to precision space via $\exp(\sum w_i \ln p_i)$.

Finally, coupling this precision term multiplicatively with the brevity penalty BP yields the complete, definitive formulation of BLEU-n:

$$\text{BLEU-}n = \mathrm{BP} \cdot \mathrm{GM}(p_1, \ldots, p_n) = \mathrm{BP} \cdot \exp \left(\sum_{i=1}^{n} w_i \ln p_i \right) = \mathrm{BP} \cdot \left(\prod_{i=1}^{n} p_i \right)^{\frac{1}{n}}. \tag{1.26}$$

Step-by-step worked example: BLEU-2 calculation.
We now execute the step-by-step computation of BLEU-2 ($n = 2$, uniform weights $w_1 = w_2 = 0.5$) on the worked example from the lecture slides.

Let the reference and machine-generated candidate sequences be:

$$R : \text{“The dog chased the cat”} \quad (r = 5 \text{ tokens}),$$
$$G : \text{“The dog ran after the cat”} \quad (g = 6 \text{ tokens}).$$

Step 1: Unigram precision ($i = 1$). Extract all unigrams from both sentences:

$$\text{Reference unigrams } R_1 = \{\text{“The”, “dog”, “chased”, “the”, “cat”}\} \quad (|R_1| = 5),$$
$$\text{Generated unigrams } G_1 = \{\text{“The”, “dog”, “ran”, “after”, “the”, “cat”}\} \quad (|G_1| = 6).$$

We count matching unigrams between G and R:

- “The”: appears in R ($\text{Count}_{\text{clip}} = 1$)
- “dog”: appears in R ($\text{Count}_{\text{clip}} = 1$)
- “ran”: absent in R ($\text{Count}_{\text{clip}} = 0$)
- “after”: absent in R ($\text{Count}_{\text{clip}} = 0$)
- “the”: appears in R ($\text{Count}_{\text{clip}} = 1$)
- “cat”: appears in R ($\text{Count}_{\text{clip}} = 1$)

Total matching unigrams = 4. The total generated unigrams = 6. The modified unigram precision is:

$$p_1 = \frac{4}{6} \approx \mathbf{0.6667}. \tag{1.27}$$

Step 2: Bigram precision ($i = 2$). Extract all contiguous pairs of adjacent tokens (bigrams):

$$\text{Reference bigrams } R_2 = \{\text{“The dog”, “dog chased”, “chased the”, “the cat”}\} \quad (|R_2| = 4),$$
$$\text{Generated bigrams } G_2 = \{\text{“The dog”, “dog ran”, “ran after”, “after the”, “the cat”}\} \quad (|G_2| = 5).$$

We count matching bigrams:

- “The dog”: matches R_2 ($\text{Count}_{\text{clip}} = 1$)
- “dog ran”: absent in R_2 ($\text{Count}_{\text{clip}} = 0$)
- “ran after”: absent in R_2 ($\text{Count}_{\text{clip}} = 0$)
- “after the”: absent in R_2 ($\text{Count}_{\text{clip}} = 0$)
- “the cat”: matches R_2 ($\text{Count}_{\text{clip}} = 1$)

Total matching bigrams = 2. The total generated bigrams = 5. The modified bigram precision is:

$$p_2 = \frac{2}{5} = \mathbf{0.4000}. \tag{1.28}$$

Step 3: Geometric mean of precisions. Combining p_1 and p_2 via the geometric mean:

$$\begin{aligned} \text{GM} &= \exp\left(\frac{1}{2}\left(\ln(0.6667) + \ln(0.4000)\right)\right) = \sqrt{p_1 \times p_2} \\ &= \sqrt{\frac{4}{6} \times \frac{2}{5}} = \sqrt{\frac{8}{30}} = \sqrt{\frac{4}{15}} \approx \mathbf{0.5165}. \end{aligned} \tag{1.29}$$

Step 4: Brevity penalty calculation. We compare the candidate length $g = 6$ against the reference length $r = 5$:

$$g = 6 > r = 5 \implies \text{BP} = 1.0. \tag{1.30}$$

Because the candidate sequence is not shorter than the reference, no brevity penalty is incurred (BP = 1.0).

Step 5: Final BLEU-2 score. Multiplying the brevity penalty by the precision geometric mean yields:

$$\text{BLEU-2} = \text{BP} \times \text{GM} = 1.0 \times 0.5165 = \mathbf{0.5165} \quad (\text{or } 51.65\%). \tag{1.31}$$

This calculation exposes the defining characteristics and underlying limitations of n-gram lexical matching. The candidate phrase "ran after" is semantically equivalent to the reference verb "chased". A human evaluator would judge "The dog ran after the cat" as a completely correct, fluent, and faithful translation. However, because BLEU enforces exact surface string equality, the bigrams "dog ran", "ran after", and "after the" were scored as total errors (0 matches). Consequently, p_2 dropped from an expected near-perfect score to 0.4000, driving the overall BLEU-2 score down to 0.5165.

Advantages and the foundational limitation of BLEU.

BLEU has served as the universal benchmark in machine translation and conditional generation for over two decades due to several compelling advantages:

- **Automated and cost-effective**: BLEU replaces slow, expensive human judges with an instantaneous, deterministic algorithmic script that costs nothing to evaluate.
- **Language-independent**: BLEU operates strictly on string matching over whitespace-separated tokens. It requires no morphological analyzers, syntactic dependency parsers, or semantic ontologies.
- **Empirical human correlation**: Across large corpora containing thousands of sentences, aggregate BLEU scores correlate reasonably well with human assessments of translation quality, effectively discriminating between high-performing and defective models.
- **Enforcement of phrasing structure**: By incorporating higher-order n-grams ($n \geq 2$), BLEU aggressively penalizes "word salad" outputs that contain relevant keywords in scrambled or ungrammatical orders.

Despite these operational strengths, BLEU exhibits a single, overarching foundational limitation: **it relies exclusively on exact surface word matching**. Every recorded failure mode and empirical deficiency of BLEU is simply a direct branch of this core constraint:

- **Synonymy and paraphrasing blindness**: Because BLEU matches character sequences rather than semantics, semantically identical phrases receive zero credit. As demonstrated in our worked example, replacing the reference verb "chased" with "ran after" destroys bigram precision despite preserving perfect semantic fidelity. Similarly, active-to-passive voice transformations and morphological inflections are penalized as complete errors.
- **Negation and factual insensitivity**: Surface character counting does not evaluate truth value. Inserting a single negation operator (e.g., generating "The system is not secure" when the reference is "The system is secure") preserves nearly all unigrams and bigrams, earning a stellar BLEU score while stating the exact opposite of the truth.
- **Brittle reference dependency**: The metric rewards adherence to the specific lexical choices of the human reference annotator. If the reference author employed uncommon vocabulary or idiosyncratic phrasing, a model generating natural, fluent, and correct prose is penalized simply because its surface tokens diverge from the human author.

- **Divergence in open-ended generation**: In tasks where generation is divergent and creative (such as dialogue, essay writing, or storytelling), there exists no unique reference sentence. Demanding exact n-gram alignment against a single reference penalizes valid linguistic creativity.

1.1.3 Recall-oriented understudy for gisting evaluation (ROUGE)

While BLEU was conceived for machine translation—where generating spurious words incurs severe penalties and precision is paramount—other natural language generation tasks require a fundamentally different optimization target. In text summarization, the primary failure mode is not verbosity, but *omission*: a candidate summary that fails to convey critical facts, characters, or conclusions is defective, even if every word it does produce matches the reference with 100% precision.

To address this challenge, Lin [**?**] introduced **ROUGE** (*Recall-Oriented Understudy for Gisting Evaluation*). In contrast to BLEU's precision-centric design, ROUGE evaluates generative performance primarily through **recall**, measuring the fraction of reference n-grams and structural subsequences successfully captured in the machine-generated hypothesis.

ROUGE-N: N-gram recall.

Let $R = \{r_1, r_2, \ldots, r_m\}$ denote a set of reference summaries, and let G denote the model-generated candidate summary. For a chosen n-gram length $n \geq 1$, ROUGE-N computes the ratio of matched n-grams over the total n-grams appearing in the reference texts:

$$\text{ROUGE-}N = \frac{\sum_{r \in R} \sum_{g_n \in r} \text{Count}_{\text{match}}(g_n)}{\sum_{r \in R} \sum_{g_n \in r} \text{Count}(g_n)}, \tag{1.32}$$

where $\text{Count}_{\text{match}}(g_n)$ represents the maximum number of times the candidate n-gram g_n co-occurs in the candidate text G and the reference r.

- **ROUGE-1 ($n = 1$)**: Measures unigram recall, quantifying whether the candidate summary contains the key thematic entities, terminology, and content words present in the human gold standard.
- **ROUGE-2 ($n = 2$)**: Measures bigram recall, testing whether vocabulary is assembled with local syntactic cohesion and pairwise grammatical fluency.

ROUGE-L: Longest common subsequence.

A limitation of fixed n-gram matching is that it requires choosing an arbitrary context window n. To capture flexible, sentence-level syntactic structure without imposing fixed n-gram boundaries, Lin [**?**] introduced **ROUGE-L**, based on the **Longest Common Subsequence** (LCS).

Given two sequences $X = (x_1, \ldots, x_m)$ and $Y = (y_1, \ldots, y_n)$, a sequence $Z = (z_1, \ldots, z_k)$ is a common subsequence if Z can be derived by deleting zero or more elements from both X and Y without altering the relative ordering of the remaining tokens. Unlike n-grams, the tokens in an LCS do not need to be contiguous.

Let R denote a reference sequence of length $r = |R|$, and let G denote a candidate sequence of length $g = |G|$. The LCS-based recall (R_{LCS}), precision (P_{LCS}), and F-measure (F_{LCS}) are defined as:

$$R_{\text{LCS}} = \frac{\text{LCS}(G, R)}{r}, \tag{1.33}$$

$$P_{\text{LCS}} = \frac{\text{LCS}(G, R)}{g}, \tag{1.34}$$

$$F_{\text{LCS}} = \frac{(1 + \beta^2) R_{\text{LCS}} P_{\text{LCS}}}{R_{\text{LCS}} + \beta^2 P_{\text{LCS}}}, \tag{1.35}$$

where β controls the relative weighting between precision and recall. In summarization benchmarks, β is typically set to a large value (such as $\beta = 1.2$ or $\beta \to \infty$) to ensure that recall dominates the evaluation.

Structural advantage of ROUGE-L. Because LCS requires tokens to appear in the same relative order, ROUGE-L naturally penalizes scrambled sentences without requiring predefined n-gram orders. If a candidate retains the subject-verb-object progression of the reference with intermediate modifiers deleted, LCS assigns a high score, reflecting accurate gist extraction.

ROUGE-W: Weighted longest common subsequence.

A structural blind spot of standard LCS is that it treats all common subsequences equally, regardless of whether their matching tokens are contiguous or widely separated across the text. For example, given the reference $R = (w_1, w_2, w_3, w_4, w_5)$, candidate $G_1 = (w_1, w_2, w_3, y_a, y_b)$ and candidate $G_2 = (w_1, y_a, w_2, y_b, w_3)$ both yield LCS = 3. However, G_1 possesses continuous phrasing, whereas G_2 is severely fragmented. **ROUGE-W** assigns higher weight to contiguous matching subsequences by applying a polynomial weight function $f(k) = k^2$ to consecutive matches.

Task-aligned metric selection: Precision in translation versus recall in summarization.

The structural divergence between BLEU and ROUGE underscores a fundamental principle of generative evaluation: metric design must mirror the primary failure mode of the downstream task. In **machine translation**, the cardinal sin is introducing spurious, fabricated, or ungrounded words that distort the original semantic meaning of the source sentence; precision is paramount, and fluent syntax is strictly monitored by higher-order n-gram precision, making BLEU the natural operational choice. Conversely, in **text summarization**, the primary failure mode is information omission—generating a grammatically flawless snippet that fails to mention the main event, characters, or conclusions renders the summary useless; recall is paramount, making ROUGE the appropriate instrument. Nevertheless, both metrics remain fundamentally bound to surface lexical matching, remaining equally blind to semantic synonymy, paraphrasing, and negation inversion.

1.1.4 Metric for evaluation of translation with explicit ordering (METEOR)

To overcome the rigid surface-matching constraints of BLEU and ROUGE, Banerjee and Lavie [**?**] developed **METEOR** (*Metric for Evaluation of Translation with Explicit ORdering*). METEOR introduces two fundamental innovations: **multi-stage linguistic matching** (incorporating morphological stemming and semantic synonymy) and an **explicit fragmentation penalty** to measure structural coherence.

The four-stage linguistic alignment cascade.

METEOR aligns candidate tokens against reference tokens by constructing a generalized bipartite matching where each token matches at most one token in the other sequence. The alignment is resolved through four successive linguistic modules:

1. **Exact match**: Pairs identical surface character strings (e.g., *chased* ↔ *chased*).
2. **Stem match**: Applies the Porter stemmer [**?**] to match words sharing identical morphological roots (e.g., *chased* ↔ *chasing*, *ran* ↔ *running*).
3. **Synonym match**: Queries WordNet synsets to match semantically equivalent terms (e.g., *chased* ↔ *pursued*, *cat* ↔ *feline*).
4. **Paraphrase match**: Matches multi-word phrase equivalents identified through statistical phrase tables.

Harmonic score and fragmentation penalty.

Once the alignment is finalized, unigram precision (P) and unigram recall (R) are computed

over the aligned matches. METEOR combines them via a parameterized harmonic mean F_{mean} that explicitly prioritizes recall:

$$F_{\text{mean}} = \frac{10\,P\,R}{R + 9\,P} = \frac{P\,R}{\alpha\,P + (1-\alpha)\,R}, \quad \text{with } \alpha = 0.9. \tag{1.36}$$

To penalize scrambled syntax, METEOR groups the matched tokens into the minimum number of adjacent contiguous blocks, called **chunks** (ch). If candidate and reference share identical word order, all matched words form a single contiguous chunk (ch = 1). If the words are randomly permuted, each word constitutes an isolated chunk (ch = m, where m is the total number of matched unigrams). The **fragmentation penalty** (Pen) is defined as:

$$\text{Pen} = \gamma \left(\frac{\text{ch}}{m}\right)^{\beta}, \quad \text{typically } \gamma = 0.5,\ \beta = 3. \tag{1.37}$$

The final METEOR score is computed as:

$$\text{METEOR} = F_{\text{mean}} \cdot (1 - \text{Pen}). \tag{1.38}$$

When syntax is preserved (ch = 1), the penalty Pen ≈ 0 and METEOR $\approx F_{\text{mean}}$. When syntax is scrambled, Pen $\to 0.5$, cutting the score in half.

Advantages and limitations of METEOR.

By incorporating stemming and synonymy, METEOR achieves substantially higher correlation with human judgments than BLEU on sentence-level translation benchmarks. However, it incurs significant trade-offs:

- **Resource dependency**: It requires curated external resources (Porter stemmers, WordNet synsets, paraphrase tables), making it difficult to port to low-resource languages.
- **Computational cost**: Constructing multi-stage graph matchings is computationally slower than computing raw n-gram intersections.

1.1.5 Contextual embedding similarity (BERTScore)

Despite the additions of stemming and WordNet lookups in METEOR, all string-matching metrics share a common Achilles' heel: **context insensitivity**. Surface strings possess polysemy; a word like "bank" refers to a financial institution or a river shore depending entirely on surrounding syntax. A static lexicon cannot distinguish context-dependent meaning.

In 2020, Zhang, Kishore, Wu, Weinberger, and Artzi [**?**] introduced **BERTScore**, transitioning evaluation from discrete character matching into continuous contextual vector spaces. Rather than matching strings, BERTScore evaluates generated text by computing pairwise cosine similarities between deep contextual embeddings generated by models such as BERT [**?**], RoBERTa, or DeBERTa.

Mathematical formulation and greedy bipartite matching.

Let $R = (x_1, x_2, \ldots, x_k)$ denote a reference sequence of k tokens, and let $G = (y_1, y_2, \ldots, y_m)$ denote a candidate generated sequence of m tokens. Both sequences are processed through a pre-trained contextual transformer, producing sequences of contextual vectors:

$$\mathbf{x}_i = \text{ContextualEncoder}(R)_i \in \mathbb{R}^d, \quad \text{for } i \in \{1, \ldots, k\}, \tag{1.39}$$

$$\mathbf{y}_j = \text{ContextualEncoder}(G)_j \in \mathbb{R}^d, \quad \text{for } j \in \{1, \ldots, m\}. \tag{1.40}$$

Every vector is normalized onto the unit hypersphere: $\|\mathbf{x}_i\|_2 = \|\mathbf{y}_j\|_2 = 1$. The pairwise similarity between reference token x_i and candidate token y_j is evaluated via the inner product (cosine similarity):

$$\mathbf{S}_{i,j} = \mathbf{x}_i^{\top} \mathbf{y}_j = \cos(\mathbf{x}_i, \mathbf{y}_j) \in [-1, 1]. \tag{1.41}$$

Instead of solving an expensive global assignment problem, BERTScore performs **greedy bipartite matching**: each token in one sequence is matched to the single most similar token in the other sequence.

- **Recall ($R_{\mathbf{BERT}}$)**: For each reference token x_i, find the maximum similarity across all generated tokens y_j, and average over reference length k:

$$R_{\text{BERT}} = \frac{1}{|R|} \sum_{x_i \in R} \max_{y_j \in G} \mathbf{x}_i^\top \mathbf{y}_j. \tag{1.42}$$

- **Precision ($P_{\mathbf{BERT}}$)**: For each generated token y_j, find the maximum similarity across all reference tokens x_i, and average over candidate length m:

$$P_{\text{BERT}} = \frac{1}{|G|} \sum_{y_j \in G} \max_{x_i \in R} \mathbf{x}_i^\top \mathbf{y}_j. \tag{1.43}$$

- **F_1-measure ($F_{\mathbf{BERT}}$)**: Synthesizes precision and recall via the harmonic mean:

$$F_{\text{BERT}} = 2 \cdot \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}. \tag{1.44}$$

Step-by-step worked numerical calculation from lecture.
To illustrate how BERTScore resolves the semantic failure modes of BLEU, we reproduce the exact numerical evaluation delivered by Prof. Giobergia during lecture.

Consider the identical reference and candidate pair analyzed in Section 1.1.2:

$$R : \text{“the dog chased the cat”} \quad (|R| = 5 \text{ tokens}),$$
$$G : \text{“the dog ran after the cat”} \quad (|G| = 6 \text{ tokens}).$$

Under BLEU-2, this pair suffered a catastrophic drop to 0.5165 because “ran after” did not match “chased” at the surface level. Table 1.3 displays the complete 6×5 pairwise cosine similarity matrix $\mathbf{S}$ computed from contextual BERT embeddings.

Table 1.3: Pairwise cosine similarity matrix $\mathbf{S}_{j,i} = \mathbf{y}_j^\top \mathbf{x}_i$ between Generated tokens (rows G) and Reference tokens (columns R) from lecture slides 17–19. Row maxima indicate candidate precision matches; column maxima indicate reference recall matches.

Candidate $G \setminus$ **Ref** R	**the** (x_1)	**dog** (x_2)	**chased** (x_3)	**the** (x_4)	**cat** (x_5)	**Row Max** ($\max_i \mathbf{S}_{j,i}$)
the (y_1)	**0.9967**	0.4120	0.3815	0.9921	0.3950	**0.9967** (matches x_1)
dog (y_2)	0.4110	**0.9968**	0.4520	0.4080	0.5820	**0.9968** (matches x_2)
ran (y_3)	0.3250	0.4610	**0.9351**	0.3180	0.4410	**0.9351** (matches x_3)
after (y_4)	0.3640	0.4100	**0.8868**	0.3720	0.3910	**0.8868** (matches x_3)
the (y_5)	0.9921	0.4080	0.3790	**0.9921**	0.3980	**0.9921** (matches x_4)
cat (y_6)	0.3920	0.5810	0.4430	0.3980	**0.9960**	**0.9960** (matches x_5)
Col Max ($\max_j \mathbf{S}_{j,i}$)	**0.9967**	**0.9968**	**0.9351**	**0.9921**	**0.9960**	—

Step 1: Precision calculation (P_{BERT}). We evaluate the maximum similarity across columns for

each generated token row:

$$\begin{aligned}
\text{Row 1 (the)} &\to 0.9967 \\
\text{Row 2 (dog)} &\to 0.9968 \\
\text{Row 3 (ran)} &\to 0.9351 \quad \text{(semantically aligned with “chased”)} \\
\text{Row 4 (after)} &\to 0.8868 \quad \text{(semantically aligned with “chased”)} \\
\text{Row 5 (the)} &\to 0.9921 \\
\text{Row 6 (cat)} &\to 0.9960
\end{aligned}$$

$$\sum_{j=1}^{6} \max_i \mathbf{S}_{j,i} = 0.9967 + 0.9968 + 0.9351 + 0.8868 + 0.9921 + 0.9960 = 5.8035. \tag{1.45}$$

Averaging over the $|G| = 6$ candidate tokens yields the precision reported in lecture:

$$P_{\text{BERT}} = \frac{5.8035}{6} \approx \mathbf{0.96725}. \tag{1.46}$$

Step 2: Recall calculation (R_{BERT}). We evaluate the maximum similarity across rows for each reference token column:

$$\begin{aligned}
\text{Col 1 (the)} &\to 0.9967 \\
\text{Col 2 (dog)} &\to 0.9968 \\
\text{Col 3 (chased)} &\to 0.9351 \quad \text{(best matched by candidate “ran”)} \\
\text{Col 4 (the)} &\to 0.9921 \\
\text{Col 5 (cat)} &\to 0.9960
\end{aligned}$$

$$\sum_{i=1}^{5} \max_j \mathbf{S}_{j,i} = 0.9967 + 0.9968 + 0.9351 + 0.9921 + 0.9960 = 4.9167. \tag{1.47}$$

Averaging over the $|R| = 5$ reference tokens yields:

$$R_{\text{BERT}} = \frac{4.9167}{5} \approx \mathbf{0.98334}. \tag{1.48}$$

Step 3: Harmonic F_1 score.

$$F_{\text{BERT}} = 2 \cdot \frac{0.96725 \times 0.98334}{0.96725 + 0.98334} = \frac{1.90220}{1.95059} \approx \mathbf{0.97519}. \tag{1.49}$$

Critical pedagogical analysis: Strengths and the “BERT says so” bottleneck.
Comparing BLEU-2 (51.65%) against BERTScore (97.52%) demonstrates the expressive power of contextual representations:

- **Paraphrase and synonym recognition**: The verbal predicate “ran after” matches “chased” with high similarity (cos $\approx$ 0.9351 and 0.8868). The model is not penalized for natural vocabulary variation.
- **Contextual disambiguation**: Because embeddings are produced by multi-head self-attention layers, each token vector encodes sentence syntax.

However, as emphasized by Prof. Giobergia, BERTScore introduces serious methodological challenges:

1. **Opacity and lack of interpretability ("BERT says so")**: If BLEU produces a low score, an engineer can inspect the exact missing n-grams. If BERTScore produces a score of 0.84, there is no symbolic, explainable justification: the score exists purely because a multi-billion-parameter neural network generated latent vectors with a specific cosine angle.

2. **Computational overhead**: Evaluating BLEU requires simple hash-map lookups taking microseconds. BERTScore requires running two complete forward passes through a transformer encoder per sentence pair, making it computationally heavy for million-sample evaluations.

3. **Inherited model bias**: Any failure mode, linguistic blind spot, or domain shift present in the evaluator model directly corrupts the evaluation metric.

1.1.6 Exact match and information retrieval ranking metrics

In tasks where the target output space is strictly bounded, continuous overlap scores are neither necessary nor desirable. When evaluating closed-domain question answering, factual entity retrieval, or programmatic code execution, a prediction is either entirely correct or unusable.

Exact match (EM).

The **Exact Match** (EM) metric evaluates binary equality between the generated hypothesis string G and the gold-standard reference string R:

$$\mathrm{EM}(G, R) = \begin{cases} 1 & \text{if Normalize}(G) = \text{Normalize}(R), \\ 0 & \text{otherwise.} \end{cases} \tag{1.50}$$

To prevent trivial punctuation or casing discrepancies from registering as false errors, the Normalize$(\cdot)$ function standardizes strings by:

- Converting all text to lowercase.
- Stripping whitespace, line breaks, and punctuation marks.
- Removing leading/trailing stop words or English articles (*a*, *an*, *the*).

Exact Match serves as the standard evaluation standard on extractive QA benchmarks (such as SQuAD [**?**]) and mathematical datasets (such as GSM8K [**?**]).

Information retrieval and probability ranking metrics.

When a language model is evaluated on multiple-choice questions, cloze passage completion, or entity ranking, it produces a probability distribution over a set of candidates $C = \{c_1, c_2, \ldots, c_K\}$. We sort the candidates by descending model probability:

$$P(c_{(1)}) \geq P(c_{(2)}) \geq \cdots \geq P(c_{(K)}).$$

Let $\mathrm{rank}_i \in \{1, 2, \ldots, K\}$ denote the position at which the ground-truth target token appears. Several canonical ranking metrics quantify performance:

- **Mean Reciprocal Rank (MRR)**: Computes the average of the reciprocal ranks of the correct answer across a query test set Q:

$$\mathrm{MRR} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\mathrm{rank}_q}. \tag{1.51}$$

  If the target item is consistently ranked first (rank = 1), MRR = 1.0. If it is ranked second, MRR = 0.5.

- **Precision at k (P@k) and Recall at k (R@k)**: Measures whether the correct answer appears within the top-k highest-probability candidates:

$$\text{Hit@}k = \mathbb{I}(\text{rank} \leq k). \tag{1.52}$$

- **Normalized Discounted Cumulative Gain (**NDCG@k**)**: When answers have varying degrees of graded relevance $rel_i \in \mathbb{R}_{\geq 0}$, NDCG@$k$ penalizes relevant items that appear lower in the ranked list using a logarithmic position discount:

$$\text{DCG@}k = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)}, \quad \text{NDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}, \tag{1.53}$$

  where IDCG@k represents the Ideal DCG obtained under a perfect sorting of relevance values.

1.1.7 Task-specific classification metrics

Many modern language model evaluations frame generative tasks as discrete classification problems by prompting the model to output a single categorical label (such as positive/negative sentiment, or entailment/contradiction in inference tasks). In these configurations, classical machine learning evaluation metrics are directly applicable:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}, \tag{1.54}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{1.55}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}, \tag{1.56}$$

where TP, TN, FP, FN denote True Positives, True Negatives, False Positives, and False Negatives, respectively.

Multi-class aggregation. For multi-class tasks (such as 3-way natural language inference), scores are aggregated using either **Macro-**F_1 (unweighted average across class F_1 scores, which prevents dominant majority classes from concealing minority class failures) or **Micro-**F_1 (global pooling of instances, identical to overall accuracy).

1.1.8 Human qualitative evaluation and model-based judges

Despite the sophistication of n-gram matching, LCS alignment, and contextual embeddings, automated metrics consistently fail to evaluate high-level cognitive properties of natural language:

- **Global coherence and argumentative structure**: Does an essay or multi-paragraph response follow a logical sequence of reasoning, or does it wander between disconnected points?
- **Factuality and hallucination detection**: A generated passage can exhibit high BLEU, ROUGE, and BERTScore overlap against a reference while introducing an unmentioned, completely false fact.
- **Creativity, tone, and stylistic appropriateness**: Measuring humor, empathy, or instruction-following fidelity cannot be reduced to cosine similarity or token overlap.

Consequently, human evaluation remains the gold standard for assessing text quality.

Human evaluation methodologies: Absolute scales versus pairwise comparisons.
Two primary paradigms govern human evaluation in research and industry:

1. **Absolute Likert rating scales**: Annotators are presented with a generated response and asked to assign a score on a discrete 1-to-5 scale across specific criteria (e.g., fluency, coherence, relevance, factual accuracy).

2. **Pairwise comparative evaluation (A/B testing)**: Annotators are presented with a prompt and two anonymized, randomized candidate outputs (A and B) produced by competing models, and asked to answer: *"Which response is better, or are they tied?"*

The cognitive failure mode of absolute scoring.
As emphasized in lecture, humans are notoriously poor at assigning absolute grades. If ten human judges are asked to grade an essay on a 1-to-5 scale, their scores exhibit severe variance due to:

- **Calibration drift**: Annotator standards shift across time due to fatigue, mood, or exposure to previous high- or low-quality samples.
- **Subjective anchor disparity**: One annotator's "4" may correspond to another annotator's "2".

In contrast, humans are remarkably consistent at **relative pairwise comparisons**: deciding whether response A is more helpful, coherent, or factually sound than response B is a significantly simpler cognitive task. Pairwise preferences are converted into global scalar rankings using the **Elo rating system [?]** (the foundation of modern public benchmarks like the LMSYS Chatbot Arena).
The scalability bottleneck and the transition to LLM-as-a-judge.
While human evaluation provides the highest qualitative fidelity, it is fundamentally bottlenecked by resource constraints:

- **Extreme financial cost**: Conducting statistically significant human trials across thousands of prompts requires hundreds of thousands of dollars in crowd-sourcing compensation.
- **High latency**: Collecting human annotations takes days or weeks, making it impossible to integrate into real-time CI/CD model training pipelines.
- **Non-reproducibility**: Human pool variance makes exact experimental replication difficult across research labs.

These constraints have catalyzed the widespread adoption of **LLM-as-a-judge [?]**. By supplying a state-of-the-art foundation model (such as GPT-4) with detailed evaluation rubrics, system instructions, and few-shot calibration examples, automated systems can evaluate model outputs pairwise with high correlation ($\rho > 0.8$) to human expert preferences, operating at fraction-of-a-cent costs and second-level latency.

While metrics provide the mathematical scoring instruments, they cannot exist in an operational vacuum. A metric like BLEU is meaningless without the task of Machine Translation; Exact Match is meaningless without Question Answering; Accuracy is meaningless without Natural Language Inference. To understand how foundational models are developed and validated, we must examine the formal problem formulations across which these metrics are deployed.

1.2 Core evaluation tasks and problem formulations

Evaluating foundation models requires probing a wide spectrum of linguistic, semantic, and reasoning capabilities. In early NLP literature, models were trained and evaluated on isolated, specialized tasks using task-specific heads. In modern foundation model engineering, diverse tasks are unified under the shared autoregressive interface: every problem is formulated as a text prompt conditioning the generation of target tokens.

This section systematically reviews the primary task paradigms covered in the course, analyzing their structural formulation, canonical datasets, and evaluation metrics.

1.2.1 Autoregressive text completion and story cloze tasks

The most direct evaluation of an autoregressive language model is **text completion**: given an initial prompt or narrative passage, the model must predict a plausible, grammatically coherent, and logically consistent continuation. This can be evaluated in two operational modes:

1. **Generative mode**: The model autoregressively generates text, which is scored via perplexity or human qualitative inspection.
2. **Discriminative choice mode**: The model is presented with a context prefix and a discrete set of competing candidate continuations, evaluating the conditional log-likelihood assigned to each continuation:

$$\hat{c} = \arg\max_{c \in C} \frac{1}{|c|} \sum_{i=1}^{|c|} \ln P(w_i^{(c)} \mid \text{Context}, w_{<i}^{(c)}). \tag{1.57}$$

Canonical benchmark datasets:

- **LAMBADA (Language Modeling Broadened to Account for Discourse-Level Analysis) [?**]: A corpus of narrative passages specifically constructed such that human readers can guess the final target word accurately when reading the entire paragraph, but cannot guess it if exposed only to the final sentence. This benchmark directly measures the model's ability to retain long-range discourse context across dozens of tokens without succumbing to recency bias. Evaluated via Accuracy (Exact Match on the final target word) and Perplexity.
- **ROCStories and StoryCloze [?**]: A benchmark composed of 5-sentence commonsense stories capturing everyday events. The StoryCloze test presents the first four sentences as context and provides two candidate endings (one logical, one absurd or contradictory). Models are evaluated on their accuracy in assigning higher likelihood to the correct narrative resolution.
- **HellaSwag [?**]: A continuation benchmark designed to counter statistical shortcut learning. Early models easily achieved high scores on story completion by exploiting shallow surface heuristics (e.g., matching length or common idioms). Zellers et al. introduced **Adversarial Filtering (AF)**, iteratively generating negative endings from a generator model that trick baseline discriminators while remaining obviously incorrect to humans.

1.2.2 Question answering architectures: Open-book versus closed-book

Question Answering (QA) evaluates a model's ability to comprehend queries, retrieve knowledge, and synthesize truthful, accurate answers. As categorized in lecture, QA problem settings differ across two foundational axes: **access to external knowledge** and **answer formulation format**.

Axis 1: Access to knowledge (Closed-book versus open-book).

- **Closed-Book QA**: The model is given only the question prompt in its context window (x = "What year was the University of Turin founded?"). The model must retrieve the factual answer entirely from its internal **parametric memory**—the weights optimized during pre-training. This evaluates factual retention and internal world models.
- **Open-Book QA**: The model is provided with external reference documents in its context window alongside the question. In modern systems, this is realized via **Retrieval-Augmented Generation (RAG)**, where an external retriever queries a document store or vector database (e.g., Wikipedia) and injects relevant context chunks into the prompt. This evaluates reading comprehension and contextual synthesis.

Axis 2: Answer format (Extractive, abstractive, and multiple-choice).

- **Extractive QA**: The answer is guaranteed to exist as a contiguous substring (span) within the provided context passage. The model predicts the starting and ending token indices.
- **Abstractive QA**: The model must generate a free-form natural language response, paraphrasing and synthesizing information across multiple sentences.
- **Multiple-Choice QA**: The model chooses among discrete candidate options (e.g., A, B, C, D).

Canonical QA benchmarks:

- **SQuAD (Stanford Question Answering Dataset)** [**?**]: Contains over 100,000 question-answer pairs curated by crowd-workers on Wikipedia articles. Answers are exact extractive text spans. SQuAD 2.0 [**?**] introduced over 50,000 *unanswerable questions* that look syntactically plausible but cannot be answered from the text, penalizing models that hallucinate answers instead of outputting an abstention signal. Evaluated via Exact Match (EM) and token-level F_1.
- **TriviaQA** [**?**]: Over 650,000 question-answer pairs compiled from trivia websites paired with independently retrieved web and Wikipedia evidence documents, introducing natural linguistic noise and distant supervision.
- **Natural Questions (NQ)** [**?**]: A dataset released by Google comprising real, uncurated user queries entered into the Google search engine, paired with full Wikipedia pages. Annotators identified both a "long answer" (an entire paragraph containing the explanation) and a "short answer" (a concise entity or span).
- **WebQuestions** [**?**]: Contains 6,642 question-answer pairs generated by querying the Google Suggest API and annotating answers via Amazon Mechanical Turk over Freebase knowledge graph entities.

1.2.3 Conditional sequence generation: Translation and summarization

Conditional sequence generation tasks require mapping an input source text into an output text with different surface syntax or different length, while preserving invariant core semantics.
Machine translation.
Machine translation maps a source sequence $\mathbf{x} = (x_1, \ldots, x_S)$ in language $\mathcal{L}_S$ to a target sequence $\mathbf{y} = (y_1, \ldots, y_T)$ in language $\mathcal{L}_T$. The premier competitive venue is the annual **WMT (Workshop**

on Machine Translation) benchmarks, which provide standardized training corpora, validation splits, and human reference sets across dozens of high-resource and low-resource language pairs (e.g., English-German, English-Czech). Evaluation relies on BLEU, METEOR, and neural metrics like COMET.

Text summarization.

Text summarization condenses an input document $\mathbf{x}$ into a concise summary $\mathbf{y}$ ($|\mathbf{y}| \ll |\mathbf{x}|$). It is divided into:

- **Extractive summarization**: Ranking and selecting the most salient verbatim sentences from the source document.
- **Abstractive summarization**: Rewriting and paraphrasing key ideas into novel, fluid sentences. Foundation models excel at abstractive summarization.

Standard benchmarks include:

- **CNN/Daily Mail [?]**: Over 300,000 news articles paired with multi-sentence bullet-point story highlights.
- **PubMed Diabetes**: Long-form scientific and biomedical research articles paired with author abstracts, requiring technical jargon retention and factual rigor.

Evaluated via ROUGE-1, ROUGE-2, and ROUGE-L.

1.2.4 Natural language inference and grammatical acceptability

Natural language inference (NLI).

NLI (also termed *recognizing textual entailment*) is the foundational benchmark for formal semantic reasoning. Given two sentences—a **Premise** (P) and a **Hypothesis** (H)—the model must determine whether the hypothesis is:

$$\text{Relationship}(P, H) \in \{\textbf{Entailment}, \textbf{Contradiction}, \textbf{Neutral}\}. \tag{1.58}$$

- *Entailment*: If P is true, H is necessarily true (P = "A dog is chasing a tennis ball", H = "An animal is running").
- *Contradiction*: If P is true, H is definitely false (P = "A man is sleeping on a couch", H = "A man is running a marathon").
- *Neutral*: H might or might not be true; P provides insufficient evidence (P = "Two people are eating pizza", "The pizza has mushrooms").

Canonical benchmarks include **SNLI (Stanford NLI) [?]** (570,000 sentence pairs derived from image captions) and **MultiNLI (Multi-Genre NLI) [?]** (433,000 sentence pairs spanning 10 diverse spoken and written genres).

Grammatical acceptability and linguistic competence.

To assess whether a model has internalized syntax independent of semantic plausibility (exemplified by Noam Chomsky's famous formulation *"Colorless green ideas sleep furiously"*), researchers utilize the **Corpus of Linguistic Acceptability (CoLA) [?]**. CoLA contains 10,657 English sentences drawn from theoretical linguistics publications, annotated as grammatically acceptable (1) or unacceptable (0). Because grammatical errors are subtle (e.g., auxiliary inversion, island constraints), CoLA is evaluated using the **Matthews Correlation Coefficient (MCC)**, a balanced measure for binary classification robust to class imbalance.

1.2.5 Text and sentiment classification

Text classification evaluates a model's capacity to categorize documents into discrete semantic classes based on overall content, style, or author intent.

- **Sentiment analysis**:
  - **IMDb Movie Reviews [?]**: 50,000 long-form movie reviews labeled with binary polarity (positive versus negative).
  - **Yelp Review Dataset**: 7 million customer reviews annotated on a 1-to-5 star rating scale.
  - **SST-2 (Stanford Sentiment Treebank) [?]**: Binary sentiment sentences derived from movie reviews, equipped with compositional syntactic parse trees.
- **Topic classification**:
  - **AG News**: 120,000 news articles categorized into four topical classes: *World*, *Sports*, *Business*, and *Sci/Tech*.
  - **20 Newsgroups**: Classic benchmark of 20,000 Usenet discussion forum messages partitioned across 20 distinct technical and social newsgroups.

Evaluated via Accuracy and Macro-F_1.

1.2.6 Mathematical and formal reasoning

Autoregressive models trained exclusively on next-token prediction frequently struggle with rigorous multi-step deductive reasoning. Small models memorize common arithmetic tables (e.g., $2 + 2 = 4$) but fail when required to execute multi-step symbolic manipulations.

To benchmark quantitative reasoning, researchers established two premier datasets:

- **GSM8K (Grade School Math 8K) [?]**: 8,500 linguistically diverse grade-school math word problems requiring 2 to 8 steps of arithmetic operations. Each problem includes a detailed chain-of-thought solution with intermediate calculation annotations (delimited by «...»). Foundation models are prompted using few-shot Chain-of-Thought (CoT) [?], and evaluated on **Exact Match** of the final numerical answer.
- **MATH Dataset [?]**: 12,500 challenging problems gathered from high-school American Mathematics Competitions (AMC 10, AMC 12, AIME), formatted entirely in LaTeX. Spans seven mathematical subfields: *Prealgebra*, *Algebra*, *Number Theory*, *Counting & Probability*, *Geometry*, *Intermediate Algebra*, and *Precalculus*, stratified across 5 difficulty levels.

1.2.7 Commonsense and causal reasoning

Commonsense reasoning represents the broad repository of unstated physical, social, temporal, and causal knowledge that humans utilize to navigate daily interactions.

Ambiguity resolution: The Winograd schema challenge.

Pronoun ambiguity resolution tests whether a model understands the physical and situational constraints of entities. The foundational benchmark is the **Winograd Schema Challenge (WSC) [?]**, based on paired sentences called *twin sentences* that differ by only one or two words:

> **Sentence 1**: *The trophy didn't fit in the suitcase because **it** was too **big**.* $\Longrightarrow$ What was too big? $\Longrightarrow$ **The trophy**.
> **Sentence 2**: *The trophy didn't fit in the suitcase because **it** was too **small**.* $\Longrightarrow$ What was too small? $\Longrightarrow$ **The suitcase**.

A model cannot resolve "it" through grammatical parsing alone; it must understand that containers are constrained by volume and that objects must be smaller than containers to fit inside them.

WinoGrande [**?**] scaled this paradigm to 44,000 adversarially filtered problems, systematically eliminating lexical selection biases to expose models relying on associative correlations rather than physical reasoning.

Causal reasoning: COPA and ATOMIC.

- **COPA (Choice of Plausible Alternatives)** [**?**]: 1,000 questions composed of a premise and two alternative hypotheses. The prompt explicitly asks for either the **cause** or the **effect**:

  Premise: The man broke his toe. What was the **cause**?
  Alternative 1: He dropped a hammer on his foot. [Correct]
  Alternative 2: He tied his shoelaces too tightly.

- **ATOMIC (An Atlas of Machine Commonsense for If-Then Reasoning)** [**?**]: A knowledge graph containing 877,000 everyday if-then relations organized into 9 semantic dimensions spanning agent intent (*xIntent*), emotional effect (*xReact*), and consequences for others (*oEffect*).

1.2.8 Synthetic probe tasks: Isolating capabilities from memorization

A persistent difficulty in evaluating foundation models on standard public datasets is determining whether a model solved a problem through general reasoning or simply reproduced text memorized during web-scale pre-training.

To address this, researchers design **synthetic probe tasks**: algorithmically generated problems that are easy to construct programmatically, yet non-trivial for autoregressive models to solve:

- **Arbitrary multi-digit arithmetic**: Generating random large-integer operations (e.g., 4123 + 9421 = 13544) guarantees that the specific numerical pair was never observed in pre-training corpora.

- **String perturbation and character scrubbing**: Requesting a model to strip a specific punctuation symbol from an arbitrary sentence (e.g., *"Remove the $ from the sentence 'th$s is a$sentenc$e" '*). Because subword tokenizers merge characters into subword units, manipulating raw characters requires navigating token boundaries.

- **Anagram unscrambling**: Asking the model to unscramble randomly permuted words (e.g., *"aplpe"* → *"apple"*).

Synthetic tasks ensure that test data is guaranteed to be unseen, providing an untainted diagnostic probe into the model's computational capabilities.

While evaluating models on individual tasks (such as SQuAD for QA, or CoLA for grammar) reveals localized strengths and weaknesses, reporting dozens of separate accuracies is unwieldy. The research community requires holistic, standardized evaluation suites that aggregate performance across diverse tasks into unified benchmarks.

1.3 Standardized benchmark suites and the contamination crisis

Foundation models are designed as general-purpose reasoners. Evaluating a model on a single task provides an incomplete picture: a model could excel at text classification while failing catastrophically at arithmetic or causal reasoning. To establish reproducible comparisons between competing architectures and training paradigms, the NLP community introduced **standardized multi-task benchmark suites**.

However, the widespread reliance on fixed, public benchmarks has triggered an epistemological crisis in AI evaluation: **benchmark data contamination**. This section examines the canonical evaluation suites and formalizes the contamination phenomenon that threatens the validity of modern LLM leaderboards.

1.3.1 Holistic multi-task benchmarking: GLUE and SuperGLUE

The GLUE benchmark.
In 2018, Wang et al. [**?**] introduced **GLUE** (*General Language Understanding Evaluation*), an integrated benchmark suite designed to evaluate general-purpose sentence representations. GLUE combined nine diverse natural language understanding tasks spanning three broad families:

1. **Single-sentence tasks**: Linguistic acceptability on CoLA [**?**] and binary sentiment analysis on SST-2 [**?**].
2. **Similarity and paraphrase tasks**: Sentence pair equivalence on MRPC (Microsoft Research Paraphrase Corpus), QQP (Quora Question Pairs), and semantic similarity scoring on STS-B (Semantic Textual Similarity Benchmark).
3. **Inference tasks**: Natural language inference on MNLI (MultiNLI) [**?**], QNLI (Question NLI, derived from SQuAD), RTE (Recognizing Textual Entailment), and WNLI (Winograd NLI).

GLUE established a public leaderboard evaluated on hidden private test sets, aggregating performance across all nine tasks into a **single summary score** (the unweighted average of task accuracies and correlation coefficients).

The saturation of GLUE and the advent of SuperGLUE.
When GLUE was introduced, state-of-the-art models scored substantially below human performance (87.1). However, with the advent of the Transformer pre-training revolution (BERT [**?**], RoBERTa, and T5 [**?**]), models rapidly saturated the benchmark, exceeding non-expert human performance within eighteen months. Analysis revealed that models were exploiting spurious statistical cues and shallow lexical overlaps in GLUE's test sets rather than exercising deep comprehension.

In response, Wang et al. [**?**] introduced **SuperGLUE** in 2019. Retaining GLUE's single-metric aggregate format, SuperGLUE replaced saturated tasks with substantially harder challenges requiring multi-sentence reasoning, coreference resolution, and open-ended QA:

- **BoolQ**: Boolean (yes/no) question answering over complex passages.
- **CB (CommitmentBank)**: Three-way discourse clause inference.
- **COPA**: Commonsense causal reasoning [**?**].
- **MultiRC**: Multi-sentence reading comprehension where questions have multiple correct answers.
- **ReCoRD**: Reading comprehension with commonsense reasoning over news articles.

- **RTE**: Recognizing textual entailment across diverse news datasets.
- **WiC (Words in Context)**: Identifying whether a polysemous word shares identical meaning across two different sentence contexts.
- **WSC**: The Winograd Schema Challenge pronoun coreference task [**?**].

1.3.2 Massive multitask language understanding (MMLU)

As foundation models scaled past tens of billions of parameters, evaluation expanded beyond basic linguistic understanding toward broad, multi-domain academic and professional knowledge.

In 2020, Hendrycks et al. [**?**] introduced **MMLU** (*Massive Multitask Language Understanding*), which has become the de facto gold standard for reporting foundation model capabilities (utilized to benchmark GPT-3, PaLM, LLaMA, GPT-4, and Claude).

Architecture and scope of MMLU:

- **Scale and format**: Comprises 15,908 multiple-choice questions, each formulated with four options (A, B, C, D).
- **Breadth of domains**: Spans **57 diverse subjects** categorized into four overarching disciplines:
  1. **STEM**: Advanced Mathematics, Physics, Chemistry, Computer Science, Machine Learning, Electrical Engineering.
  2. **Humanities**: Formal Logic, Philosophy, World History, Prehistory, Jurisprudence.
  3. **Social Sciences**: Microeconomics, Macroeconomics, Psychology, Sociology, US Government and Politics.
  4. **Other / Professional**: Clinical Medicine, Professional Law (Bar Exam), Professional Accounting, Virology.
- **Difficulty stratification**: Covers elementary school facts, high school examinations, undergraduate coursework, and professional licensing standards.

Evaluation protocol.

Models are evaluated on MMLU under **zero-shot** or **5-shot in-context learning** conditions. The prompt provides the question, four formatted choices, and a prompt prefix (e.g., *"Answer:"*). The model's score is computed by comparing the conditional log-probabilities assigned to the four option tokens:

$$\hat{y} = \underset{k \in \{\text{'A'},\text{'B'},\text{'C'},\text{'D'}\}}{\arg\max} P(w_k \mid \text{Prompt}). \tag{1.59}$$

The overall MMLU score is the unweighted percentage of questions answered correctly across all 57 subjects. A random baseline achieves 25.0%; modern frontier models (such as GPT-4 and Gemini 1.5) exceed 85%, outperforming human domain experts on the majority of specialized subjects.

Concrete benchmark question examples from lecture.

To illustrate the depth and format of MMLU across diverse disciplines, consider the representative questions highlighted in the lecture:

MMLU Example 1: Conceptual Physics

Question: When you drop a ball from rest it accelerates downward at 9.8 m/s^2. If you instead throw it downward assuming no air resistance its acceleration immediately after leaving your hand is:

(A) 9.8 m/s^2 *[Correct]*

(B) more than 9.8 m/s^2

(C) less than 9.8 m/s^2

(D) Cannot say unless the speed of throw is given

Physics insight: Once the ball leaves the hand, gravity is the only accelerating force acting on the mass; the initial throw velocity alters instantaneous speed, not downward acceleration ($g = 9.8$ m/s^2).

MMLU Example 2: College Mathematics

Question: In the complex z-plane, the set of points satisfying the equation $z^2 = |z|^2$ is a:

(A) pair of points

(B) circle

(C) half-line

(D) line *[Correct]*

Mathematical derivation: Let $z = x + iy$. Then $z^2 = x^2 - y^2 + 2ixy$ and $|z|^2 = x^2 + y^2$. Equating real and imaginary components yields $2xy = 0$ (so $x = 0$ or $y = 0$) and $x^2 - y^2 = x^2 + y^2 \implies 2y^2 = 0 \implies y = 0$. Since $y = 0$ and $x \in \mathbb{R}$ satisfies the equation for all x, the solution set is precisely the real axis, which constitutes a **line** in the complex plane.

1.3.3 The benchmark contamination crisis

The astronomical performance gains reported on public benchmarks have brought to light an existential vulnerability in modern AI evaluation: **data contamination**.

Definition 1.1: *Benchmark Data Contamination*

Data contamination refers to the inadvertent or deliberate inclusion of evaluation benchmark test data (questions, passages, or gold-standard target labels) in the training corpus of a language model.

As highlighted in the lecture, Foundation models are pre-trained on multi-terabyte web crawls (such as Common Crawl, GitHub, Wikipedia, and academic preprint servers) containing hundreds of billions of web pages. Because popular benchmarks (GLUE, SQuAD, MMLU, GSM8K) are hosted on public GitHub repositories, open-access papers, and web forums, their test sets are routinely ingested by automated web scrapers during pre-training.

Taxonomy and levels of contamination:

Contamination can occur across multiple distinct stages of the model lifecycle:

1. **Pre-training contamination**: Public benchmark test splits are included directly in the pre-training scrape. The model observes the verbatim test questions and answers during self-supervised next-token prediction, memorizing them as factual text.

2. **Supervised fine-tuning (SFT) contamination**: Benchmark questions are inadvertently used to generate synthetic instruction-tuning datasets (e.g., using one LLM to generate Q&A pairs from existing benchmarks to train another model).

3. **Post-deployment feedback contamination**: Real users paste benchmark questions into public chatbot interfaces, which are subsequently collected and added to subsequent retraining pipelines.

Theoretical consequences: Memorization versus reasoning.
Data contamination undermines the foundational assumption of statistical machine learning: that generalization performance on a held-out test set reflects genuine out-of-distribution capability.

In their seminal study *"NLP Evaluation in Trouble: On the Need to Measure LLM Data Contamination for Each Benchmark"*, Sainz et al. [**?**] demonstrated that models exhibiting near-perfect scores on popular reasoning benchmarks suffered catastrophic performance drops when evaluated on slightly perturbed or synthetically generated instances of the identical problems. A model that achieves 90% on GSM8K might simply be reciting a memorized solution path observed dozens of times during pre-training, rather than executing mathematical deduction.
The closed-model auditing barrier.
As emphasized by Prof. Giobergia, data contamination introduces severe asymmetries between open-weight and proprietary closed-source models:

- For open-source models (such as LLaMA or Mistral), researchers can inspect public pre-training manifests, search for exact n-gram overlaps, and audit decontamination pipelines.

- For proprietary commercial models (such as GPT-4), the exact composition of the pre-training corpus is an undisclosed trade secret. Consequently, **contamination cannot be formally proven or disproven**. Continuous benchmark gains reported by proprietary models may stem from architectural innovations, or simply from aggressive pre-training exposure to the evaluation test sets.

Decontamination protocols and modern countermeasures.
To safeguard evaluation integrity, modern foundation model research employs rigorous decontamination heuristics:

- **N-gram overlap filtering**: Scrutinizing the pre-training corpus for contiguous 13-gram or 15-gram matches against benchmark test splits and removing matching documents prior to training.

- **Perplexity anomaly inspection**: Measuring the model's perplexity on the benchmark test set compared to a fresh, private validation set. If perplexity on the public test set is anomalously lower than on independent text from the same domain, contamination is strongly indicated.

- **Dynamic and synthetic benchmarks**: Generating novel evaluation instances on-the-fly (e.g., dynamic arithmetic problems or continuous human evaluation arenas like Chatbot Arena) that can never appear in static training crawls.

1.4 Synthesis: The language model evaluation matrix

To provide a comprehensive reference for researchers and software engineers, Table 1.4 synthesizes the entire hierarchy of language model evaluation examined in this chapter, connecting evaluation paradigms, operational metrics, canonical benchmark datasets, and primary failure modes.

Chapter conclusion.

Language model evaluation has evolved from simple probabilistic uncertainty metrics (PPL) into multi-faceted empirical benchmarking across thousands of specialized tasks. As foundation models approach and surpass human performance across standard benchmarks, evaluation methodology itself becomes the central engineering frontier: moving from static public test sets toward dynamic, synthetic, and adversarial evaluation protocols capable of discriminating between superficial web memorization and genuine machine intelligence.

Table 1.4: Comprehensive synthesis of language model evaluation paradigms, tasks, canonical datasets, standard metrics, and structural failure modes.

Paradigm	Core Task	Canonical Datasets	Standard Metrics	Primary Failure Modes
Intrinsic Likelihood	Next-token prediction	Raw text corpora	Perplexity (PPL), Cross-Entropy	Incomparable across tokenizers; rewards surface familiarity over truth.
Lexical Surface Matching	Machine translation, summarization	WMT, CNN/Daily Mail	BLEU-n, ROUGE-N, ROUGE-L, METEOR	Semantic blindness; penalizes valid paraphrasing; negation insensitivity.
Semantic Embeddings	Paraphrase generation, open dialogue	General NLG corpora	BERTScore ($P_{\text{BERT}}, R_{\text{BERT}}, F_{\text{BERT}}$)	High computational cost; opaque interpretability ("BERT says so").
Exact Match & Ranking	Extractive QA, multiple choice	SQuAD, Natural Questions	Exact Match (EM), MRR, NDCG@k, Hit@k	Zero partial credit; sensitive to superficial formatting discrepancies.
Linguistic Inference	NLI, grammatical acceptability	SNLI, MultiNLI, CoLA	Accuracy, Macro-F_1, MCC	Susceptible to lexical overlap heuristics; dataset artifact exploitation.
Multi-Step Reasoning	Grade-school & competition math	GSM8K, MATH	Exact Match on final answer, Pass@1	Calculation brittleness; superficial step hallucination; CoT drift.
Commonsense & Causality	Ambiguity resolution, if-then causality	Winograd, Winogrande, COPA, ATOMIC	Accuracy, Choice Log-Likelihood	Memorized associative priors override true physical/temporal reasoning.
Holistic Benchmarks	Multi-task understanding, professional knowledge	GLUE, SuperGLUE, MMLU	Aggregate Score, Mean Macro-F_1	Rapid benchmark saturation; severe vulnerability to data contamination.
Qualitative & Human	Open-ended chat, instruction following	Chatbot Arena, MT-Bench	Elo rating, Pairwise win rate, LLM-as-a-judge	Human judge fatigue and calibration drift; evaluator LLM verbosity and position bias.